E. The 67th XOR Problem

time limit per test 3 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1200 | Tags: binary search, bitmasks, brute force | https://codeforces.com/problemset/problem/2218/E

The PMOI wasn't even that awful - word on the street was that the problems were decent, and even (heaven forbid) enjoyable! To Macaque's dismay, however, three unruly participants (Cloud, ChatGBT and Grook) started cheating using their perfect memory of the OEIS. What utter rapscallions! Suddenly, Macaque had to act fast, and devise a problem that these three could not feasibly cheat on. You, demoted from the rank of his companion to his wretched slave, have been enlisted to test.

You are given an array a, initially containing n non-negative integers.

You perform the following operation exactly n-1 times:

1. Select an index i of a ($1 \le i \le |a|$, where $|a|$ denotes the current length of a). Let $x = a_i$.
2. Set $a_j = a_j \oplus x$ for all $1 \le j \le |a|$, where $\oplus$ denotes the bitwise XOR operation.
3. Remove a_i from the array.

It can be shown that after n-1 operations, exactly one element remains in the array. Your task is to determine the maximum possible value of this final remaining element if you perform the operations optimally.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 100$). The description of the test cases follows.

The first line of each test case contains a single integer n ($2 \le n \le 3105$) — the initial length of the array.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($0 \le a_i \le 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed 3105.

Output

For each test case, output a single integer — the maximum possible value of the final element.

Example

Input
3 2 67 67 3 1 2 3 10 67 667 167 867 267 467 367 567 767 967

Output
0 3 1012

Note

In the second test case, the array is [1, 2, 3]. One optimal sequence of operations is:

1. Select the element 3. Remove it. The remaining elements become [1 □ 3, 2 □ 3] = [2, 1].
2. Select the element 2. Remove it. The remaining element becomes [1 □ 2] = [3].

The final value is 3.